

LLM-ERP Operation Manual

LLM-Powered Enterprise Resource Planning System

Version: v0.1.0 | Updated: 2026-05-07

1. System Overview

LLM-ERP is an open-source intelligent ERP system that lets you manage your entire factory floor through **natural language** — no menu clicking, no T-codes to memorize. It covers 7 modules:

Module	Code	Core Function
 Inventory	MM	Part management, stock query, inbound/outbound transactions, location tracking
 Purchasing	PP	Supplier management, PO lifecycle, supplier scoring
 BOM Engineering	ENG	Product structure, multi-level explosion, shortage checking
 Dispatch	MFG	Work order management, machine scheduling, dynamic rescheduling (3 strategies)
 Quality	QM	Inspection orders, non-conformance tracking, CAPA
 Accounting	FI	Chart of accounts, journal entries, AR aging, month-end close
 War Room	—	SVG value-stream dashboard, real-time event animations, multi-screen display

2. Requirements

Hardware Requirements

Setup	Minimum	Recommended
RAM	4 GB	16 GB (with local LLM)
CPU	2 cores	8 cores
Local LLM	—	16 GB RAM + GPU (optional)

Software Requirements

- Python 3.11+
- Node.js 18+
- (Optional) Docker Desktop — for PostgreSQL production environment
- (Optional) Ollama — for local LLM inference

3. Installation

3.1 Download the Project

```
git clone https://github.com/fanchanyu/llm-erp.git
cd llm-erp
```

3.2 Backend Configuration

```
cd backend
cp .env.example .env
# Edit .env with your LLM Provider's API Key
```

.env File Reference:

Variable	Description	Example
LLM_PROVIDER	LLM provider	deepseek / ollama / anthropic / openai
LLM_MODEL	Model name	deepseek-chat / gemma4:e4b / claude-sonnet-4
MAX_TOOL_ROUNDS	Max tool call rounds	5 (cloud) / 8-10 (local)
DEEPSEEK_API_KEY	DeepSeek API Key	sk-xxxx

3.3 Getting an API Key (Choose One Provider)

LLM-ERP needs an LLM (Large Language Model) to power its natural language interface. Below are sign-up guides for each provider:

♦ **DeepSeek (Recommended — cheapest)**

1. Go to https://platform.deepseek.com/sign_up and register
2. Verify email → Log in
3. Click "API Keys" on the left sidebar → "Create API Key"
4. Copy the key (format: sk-xxxxxxxxxxxxxxxxxx)
5. Set in .env: DEEPSEEK_API_KEY=sk-xxxx

- **Cost:** ~~¥1 CNY processes 200~~ 500 queries
- **Model:** deepseek-chat
- **Best for:** Getting started, development phase

♦ **Anthropic Claude (Smartest)**

1. Go to <https://console.anthropic.com/> and register
2. Log in → Click "API Keys" → "Create Key"
3. Copy the key (format: sk-ant-xxxxxxxxxxxxx)
4. Set in .env:
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4
ANTHROPIC_API_KEY=sk-ant-xxxx

- **Cost:** \$3/M input + \$15/M output tokens
- **Models:** claude-sonnet-4 (recommended) / claude-haiku-3-5 (fast & cheap)
- **Best for:** Production environment, high accuracy needs

♦ **OpenAI GPT (Most popular)**

1. Go to <https://platform.openai.com/signup> and register
2. Log in → Click top-left → "API Keys" → "Create new secret key"
3. Copy the key (format: sk-proj-xxxxxxxxxxxx)
4. Set in .env:

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o
OPENAI_API_KEY=sk-proj-xxxx
```

- **Cost:** \$2.50/M input + \$10/M output (gpt-4o)
- **Models:** `gpt-4o` (recommended) / `gpt-4o-mini` (budget)
- **Best for:** Existing OpenAI users, general purpose

♦ OpenRouter (One endpoint for many models)

Provides a unified API endpoint for multiple models including DeepSeek, Claude, GPT, and more:

```
1. Go to https://openrouter.ai/keys and register
2. Log in → "Create Key"
3. Copy the key (format: sk-or-v1-xxxxxxxxxx)
4. Set in .env:
    LLM_PROVIDER=openrouter
    LLM_MODEL=deepseek/deepseek-chat
    OPENROUTER_API_KEY=sk-or-v1-xxxx
```

- **Cost:** Varies by model (you can pick the cheapest)
- **Model list:** <https://openrouter.ai/models>
- **Best for:** Comparing multiple models, easy switching

♦ Ollama (Local model, completely free, no API Key needed)

If your computer has 16GB+ RAM, you can run models locally with zero API costs:

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Download Gemma4 model (8B, ~9.6GB)
ollama pull gemma4:e4b

# Verify Ollama is running
curl http://localhost:11434/api/tags

# Set .env:
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

- **Cost:** Completely free — just need hardware
- **Hardware:** 16GB RAM (8B model) / 32GB (12B model)
- **Best for:** Data privacy, no API costs

⚠ **API Key Security: Keys must only go in `.env`. NEVER put them in CSV data files, Git commits, or post them on GitHub.**

3.8 Start the Backend

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

The backend starts at `http://localhost:8000` .

3.9 Start the Frontend

```
cd frontend
npm install
npm run dev
```

The frontend starts at `http://localhost:5173` .

3.10 Start Local LLM (Optional)

If you chose Ollama, first follow [Section 3.3 — Ollama](#) to install Ollama and download the model, then verify:

```
curl http://localhost:11434/api/tags
```

Then go back to [Section 3.8](#) to start the backend.

3.11 Data Management (Import / Export / Reset)

LLM-ERP provides a standardized data import/export tool. All data uses **natural keys** (part numbers, supplier names, product codes) — not UUIDs — so CSV files are human-readable and directly editable. **API keys are NEVER stored in data files**, only in `.env` .

Import Data

```
cd backend

# Import built-in sample data (12 tables, 71 records)
python -m scripts.manage_data import scripts/sample_data/

# Or use your own CSV
python -m scripts.manage_data import path/to/my-data.csv

# Or import an entire directory (auto-sorted by dependency order)
python -m scripts.manage_data import path/to/data-dir/

# Dry run (validate without modifying DB)
python -m scripts.manage_data import scripts/sample_data/ --dry-run
```

Export Data

```
python -m scripts.manage_data export ./backup/
# Outputs: 01-parts.csv ~ 12-ar.csv, 12 tables total
```

Reset Database

```
python -m scripts.manage_data reset --force
# Clears all 22 tables
```

View Schema

```
python -m scripts.manage_data schema
# Lists all entity field definitions
```

CSV Schema Quick Reference

File	Entity	Required Fields	Depends On
01-parts.csv	Parts Master	part_no, name, unit	—
02-suppliers.csv	Suppliers	name	—
03-products.csv	Products	product_no, name	—
04-work-centers.csv	Work Centers	name	—
05-accounts.csv	Chart of Accounts	account_no, name, type, normal_balance	—
06-inventory.csv	Stock Levels	part_no, location, quantity	parts
07-bom.csv	Bill of Materials	product_no, part_no, quantity, level	products, parts
08-purchase-orders.csv	Purchase Orders	po_no, supplier_name, item_part_no, item_quantity	suppliers, parts
09-production-orders.csv	Production Orders	order_no, product_no, quantity, due_date	products
10-quality.csv	Inspection Orders	inspection_no, part_no, quantity	parts
11-accounting.csv	Journal Entries	entry_no, description, entry_date, period, line_account_no, line_debit, line_credit	accounts
12-ar.csv	Accounts Receivable	customer_name, invoice_no, amount, due_date	—

The import tool handles dependency ordering automatically and skips existing records (idempotent).

4. Usage Guide

4.1 Inventory

Ask anything about stock:

English Query	Description
"How many M6x20 screws are in stock?"	Query specific part stock
"Show me the full inventory list"	List all stock
"Which transmission parts are low on stock?"	Category-based query
"Check motor parts stock status"	Fuzzy name search
"Receive 500 M6x20 screws"	Inbound transaction

"Issue 100 M6x20 screws to work order WO-001"	Outbound/picking
---	------------------

4.2 Purchasing

English Query	Description
"List all suppliers, check DaMing Screws"	Query suppliers
"Create a PO to DaMing Screws for 200 M6x20 screws"	Create purchase order
"List all current purchase orders"	Query PO list
"Has PO-2026-0001 arrived?"	Check PO status

4.3 BOM Management

English Query	Description
"What parts does CNC-001 use?"	Query BOM structure
"Show me the CNC-001 BOM explosion"	Multi-level BOM explosion
"Can I build 5 CNC-001 units? Check material availability"	Shortage check (core feature)

4.4 Production Dispatch

English Query	Description
"List all current work orders"	Query work orders
"Release work order WO-20260506-001"	Release work order
"CNC-01 is down, right-shift 30 minutes"	Right-shift reschedule
"CNC-01 is down, switch to backup machine"	Route change reschedule
"Emergency order WO-002, expedite it"	Expedite (insertion)

4.5 Quality Management

English Query	Description
"Create inspection for M6x20, lot LOT-001"	Create inspection order
"Show inspection records"	Query inspection list
"Are there any NCs for M6x20?"	Query non-conformances
"Create NC for M6x20, dimension out of spec, severity=Major"	Create NC record

4.6 Accounting

English Query	Description
"Which AR invoices are overdue?"	Query overdue AR
"Show me the AR aging report"	Query AR list
"List chart of accounts"	Query accounts

"Create journal entry: debit Inventory 1000 credit Bank 1000"	Create journal entry
---	----------------------

4.7 Cross-Module Queries

English Query	Description
"Has PO-0001 for M6x20 screws arrived? Has it been received?"	Purchasing + Inventory
"I need to build 5 CNC-001 units — first check if materials are available"	BOM + Purchasing

5. War Room

The War Room is a full-screen SVG value-stream dashboard, suitable for factory floor multi-monitor setups.

How to open:

```
# After starting the frontend, open in browser:
http://localhost:5173/war-room.html
```

War Room features:

- 🏢 Horizontal 6-stage layout: Supplier → Purchasing → Inventory → Dispatch → Quality → Accounting
- 🔵 Animated particle flow: Green = material flow, Yellow = cash flow
- 📡 Bottom event stream: Shows the latest 30 events in real time
- 🔔 Node glow: Event-triggered visual feedback on corresponding stage
- ▶ Simulate events button: Demo-mode auto event generation
- 🔄 Auto-refresh every 15 seconds

6. Provider Switching

LLM-ERP supports 5 LLM Providers — switch anytime:

```
# Edit backend/.env
LLM_PROVIDER=deepseek           # or ollama / anthropic / openai / openrouter
LLM_MODEL=deepseek-chat         # or gemma4:e4b / claude-sonnet-4 / gpt-4o
MAX_TOOL_ROUNDS=5               # cloud=5, local=8-10
```

Switching principles:

- Cloud API** (DeepSeek / Claude / GPT): MAX_TOOL_ROUNDS=5, needs API Key
- Local model** (Ollama / Gemma4): MAX_TOOL_ROUNDS=8-10, no API Key needed
- Backend auto-reloads on `.env` change (`uvicorn --reload`)

Provider Benchmark (30-test)

Metric	DeepSeek (Cloud)	Gemma4 8B (Local CPU)
	Pass Rate	90% (27/30)
	Avg Response	7.7s
	Cost per Test	~\$0.002
	Data Sovereignty	External API

7. Running the Benchmark

```
cd /mnt/d/Project/LLM_ERP/evaluation

# Run standard 30-test benchmark
python3 run_eval.py

# Verbose output
python3 run_eval.py --verbose

# Specify output file
python3 run_eval.py --output my-results.json
```

8. Frequently Asked Questions

Q: Backend won't start?

A: Make sure `.env` has a valid API Key. Make sure `pip install -r requirements.txt` has been run.

Q: LLM response is too slow?

A: DeepSeek averages 7.7s, which is normal. For local models (Gemma4), CPU inference takes ~8-10s; GPU can drop to 1-3s.

Q: Query results are wrong?

A: Try increasing `MAX_TOOL_ROUNDS` (local models need more rounds). You can also add specific constraints to the system prompt.

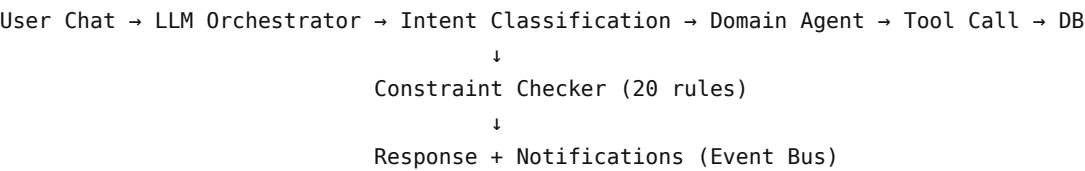
Q: How do I clear the database?

A: Use the built-in tool: `cd backend && python -m scripts.manage_data reset --force`. Or delete `backend/llm_erp.db` and re-import.

Q: Vite doesn't pick up new files in public/?

A: Restart Vite dev server: `Ctrl+C` then re-run `npm run dev`.

9. System Architecture Quick Reference



Layer	Technology	Description
Frontend	React 18 + TypeScript + Vite + Tailwind	Role dashboards, Chat UI, War Room
Backend	Python FastAPI + SQLAlchemy	7 domain services, 42 API routes
LLM	DeepSeek / Anthropic / OpenAI / Ollama	27 tool definitions, Function Calling
Events	Pub/Sub Event Bus	10 event types, role-based routing

Database	SQLite (dev) / PostgreSQL (prod)	22 tables, Alembic migrations
----------	----------------------------------	-------------------------------

This manual corresponds to LLM-ERP v0.1.0. Updated: 2026-05-07.